



Chuyên đề: Kỹ thuật Hai con trỏ & Cửa sổ trượt Nâng cao (Advanced Two Pointers & Sliding Window)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

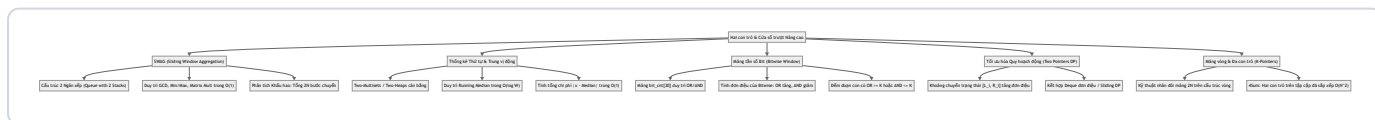


1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lập trình thi đấu cơ bản, kỹ thuật **Hai con trỏ (Two Pointers)** và **Cửa sổ trượt (Sliding Window)** thường gắn liền với bài toán tính tổng (Sum) hoặc đếm tần số ký tự — nơi phần tử rời khỏi cửa sổ có thể được "hoàn tác" dễ dàng bằng phép trừ ($S \leftarrow S - A[L]$) hoặc giảm tần số (`cnt[x]--`).

Tuy nhiên, trong các kỳ thi **ICPC, Olympic Tin học Sinh viên và Codeforces Div 2 D-F / Div 1**, chúng ta đối mặt với các bài toán cửa sổ trượt phức tạp hơn nhiều:

- Toán tử không có phép nghịch đảo (Non-invertible Monoids):** Làm thế nào để duy trì $\gcd(A[L \dots R])$, $\min / \max$, tích ma trận, hoặc phép toán bitwise OR/AND trên cửa sổ trượt co giãn trong $O(1)$ amortized mà không dùng Segment Tree $O(N \log N)$?
- Duy trì Trung vị động & Thống kê thứ tự (Order Statistics):** Cân bằng hai cấu trúc Heap/Multiset để tìm trung vị và tối ưu hóa hàm khoảng cách L_1 trực tuyến trong $O(\log W)$.
- Mảng đếm Bit (Bit Frequency Array):** Quản lý tính chất đơn điệu của OR/AND trên 30–62 bit với chi phí $O(\log(\max A))$.
- Tối ưu hóa Quy hoạch động bằng Two Pointers (Two Pointers DP):** Khai thác tính đơn điệu của điểm cực trị chuyển trạng thái $opt(i)$ để hạ bậc độ phức tạp từ $O(N^2)$ xuống $O(N)$.
- Cửa sổ trượt trên Mảng vòng (Circular Sliding Window) & Đa con trỏ (Multi-Pointers):** Mở rộng không gian trạng thái trên cấu trúc vòng và khử trùng lặp đa chiều.



2. Cấu trúc SWAG (Sliding Window Aggregation)

2.1. Vấn đề cốt lõi: Phép toán không khả nghịch (Non-invertible Operators)

Xét một phép toán hai ngôi $\odot$ có tính **kết hợp** (Associative): $(a \odot b) \odot c = a \odot (b \odot c)$ và có phần tử trung hòa e (tạo thành một Monoid).

- **Phép toán khả nghịch (Invertible):** Phép cộng (+) có phép trừ (−), phép nhân (×) có phép chia (÷). Khi

phần tử $A[L]$ trượt ra khỏi cửa sổ, ta hoàn tác bằng $Total \leftarrow Total - A[L]$ trong $O(1)$.

- **Phép toán không khả nghịch (Non-invertible)**: $\gcd(a, b)$, $\min(a, b)$, $\max(a, b)$, $a \mid b$ (bitwise OR), $a \& b$ (bitwise AND), phép nhân hai ma trận $M_1 \times M_2$.

- Nếu biết $\gcd(A[L], A[L+1], \dots, A[R])$, khi loại bỏ $A[L]$, ta **không thể** suy ra $\gcd(A[L+1 \dots R])$ chỉ bằng phép toán $O(1)$ thông thường!

- Cách tiếp cận ngây thơ dùng Segment Tree mất $O(N \log N)$ thời gian và hằng số lớn.

- **Kỹ thuật SWAG** giải quyết trọn vẹn bài toán này trong $O(1)$ **amortized** và $O(1)$ **worst-case query**!

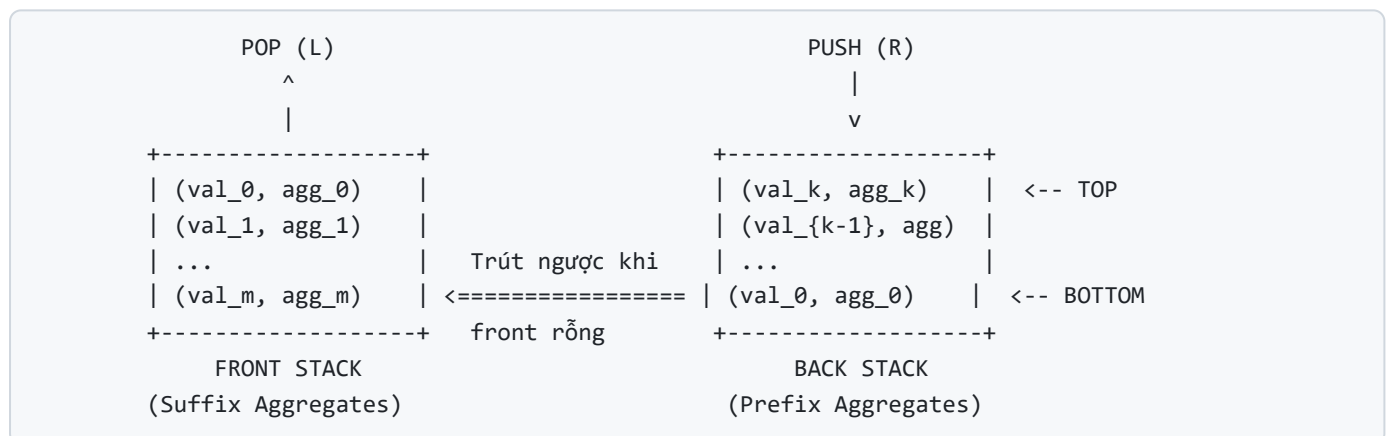
2.2. Cơ chế 2 Ngăn xếp (Queue using Two Stacks)

Để cài đặt một Hàng đợi (Queue) hỗ trợ truy vấn tổng hợp Monoid, ta sử dụng 2 Stack: `stack_front` (đầu ra) và `stack_back` (đầu vào).

Mỗi phần tử trong stack lưu một cặp: (value, aggregate), trong đó aggregate là kết quả tích lũy từ đáy stack lên đến vị trí hiện tại:

$$\text{back_stack}[k].\text{agg} = \text{back_stack}[k-1].\text{agg} \odot \text{back_stack}[k].\text{val}$$

$$\text{front_stack}[k].\text{agg} = \text{front_stack}[k].\text{val} \odot \text{front_stack}[k-1].\text{agg}$$



3 Thao tác cơ bản:

1. **Push x (Mở rộng cửa sổ - R tăng):**

2. Đẩy vào `stack_back`.

3. `new_agg = stack_back.empty() ? x : (stack_back.top().agg  $\odot$  x).`

4. Chi phí: $O(1)$.

5. **Pop (Thu hẹp cửa sổ - L tăng):**

6. Nếu `stack_front` không rỗng: Đơn giản thực hiện `stack_front.pop()`.

7. Nếu `stack_front` rỗng: Ta **trút toàn bộ** các phần tử từ `stack_back` sang `stack_front`. Khi trút, thứ tự bị đảo ngược (LIFO $\rightarrow$ FIFO). Ta tính lại `agg` cho từng phần tử khi nạp vào `stack_front` từ đáy lên.

8. Chi phí: Phân tích khấu hao $O(1)$ mỗi phần tử.

9. **Query toàn bộ cửa sổ $[L, R]$:**

10. Nếu `stack_back` rỗng: Kết quả là `stack_front.top().agg`.

11. Nếu `stack_front` rỗng: Kết quả là `stack_back.top().agg`.

12. Nếu cả hai đều có phần tử: Kết quả là:

$$\text{Total Agg} = \text{stack_front.top().agg} \odot \text{stack_back.top().agg}$$

13. Chi phí: **Chính xác** $O(1)$!

2.3. Chứng minh Phân tích Khấu hao (Amortized Analysis)

- Mỗi phần tử được nạp vào `stack_back` đúng **1 lần** ($O(1)$).
- Mỗi phần tử được chuyển từ `stack_back` sang `stack_front` đúng **1 lần** ($O(1)$).
- Mỗi phần tử bị loại khỏi `stack_front` đúng **1 lần** ($O(1)$).
- Khi trượt cửa sổ qua mảng kích thước N :

$$\text{Tổng số thao tác} \leq 3N = O(N)$$

- Do đó, chi phí khấu hao trung bình cho mỗi bước di chuyển con trỏ là $O(1)$.

💡 **So sánh:** Segment Tree mất $O(N \log N)$ thời gian và $O(4N)$ bộ nhớ với hằng số nhảy con trỏ cache lớn. SWAG chỉ tốn $O(N)$ thời gian tuyến tính, $O(N)$ mảng liên tục cực kỳ thân thiện với CPU Cache!

1234 3. Kỹ thuật Mảng tần số Bit (Bitwise Sliding Window)

3.1. Tính đơn điệu của Phép toán Bitwise

Xét dãy $A_1, A_2, \dots, A_N$ gồm các số nguyên không âm W -bit ($W \leq 60$):

- **Phép toán OR:** Khi thêm một phần tử vào cửa sổ, các bit 1 chỉ có thể tăng lên hoặc giữ nguyên, **không bao giờ giảm**:

$$\text{OR}(A[L \dots R+1]) \geq \text{OR}(A[L \dots R])$$

- **Phép toán AND:** Khi thêm một phần tử vào cửa sổ, giá trị chỉ có thể giảm đi hoặc giữ nguyên, **không bao giờ tăng**:

$$\text{AND}(A[L \dots R+1]) \leq \text{AND}(A[L \dots R])$$

Nhờ tính chất đơn điệu này, bài toán tìm đoạn con dài nhất / ngắn nhất có $\text{OR} \geq K$ hoặc $\text{AND} \leq K$ hoàn toàn thỏa mãn điều kiện áp dụng Hai con trỏ!

3.2. Mảng đếm Bit `bit_cnt[60]`

Để duy trì giá trị OR và AND của cửa sổ $[L, R]$ trong $O(W)$, ta duy trì mảng:

$\text{bit_cnt}[b] =$ Số lượng phần tử trong $[L, R]$ có bit thứ b được bật (bằng 1)

```
struct BitwiseWindow {
    int bit_cnt[62] = {0};
    int len = 0;

    void add(long long x) {
        for (int b = 0; b < 62; ++b) {
            if ((x >> b) & 1) bit_cnt[b]++;
        }
        len++;
    }

    void remove(long long x) {
        for (int b = 0; b < 62; ++b) {
            if ((x >> b) & 1) bit_cnt[b]--;
        }
        len--;
    }

    long long get_or() const {
        long long res = 0;
        for (int b = 0; b < 62; ++b) {
            if (bit_cnt[b] > 0) res |= (1LL << b);
        }
        return res;
    }

    long long get_and() const {
        if (len == 0) return 0;
        long long res = 0;
        for (int b = 0; b < 62; ++b) {
            if (bit_cnt[b] == len) res |= (1LL << b);
        }
        return res;
    }
};
```

Độ phức tạp: Mỗi thao tác `add`, `remove`, `get_or`, `get_and` chỉ mất $O(W)$ (với $W \leq 30$ cho `int`, $W \leq 62$ cho `long long`). Toàn bộ thuật toán trượt qua mảng N phần tử chỉ mất $O(N \cdot W)$ thao tác, hoàn toàn chạy dưới 0.1 giây cho $N = 2 \cdot 10^5$.

4. Thống kê Thứ tự & Duy trì Trung vị Động (Sliding Window Median)

4.1. Bài toán Trung vị và Tối ưu hàm khoảng cách L_1

Xét cửa sổ kích thước cố định K trượt qua mảng. Tại mỗi vị trí, ta cần tìm:

- Giá trị **Trung vị (Median)** của các phần tử trong cửa sổ.
- Chi phí tối thiểu** để biến đổi tất cả các phần tử trong cửa sổ về cùng một giá trị c :

$$\min_c \sum_{i=L}^R |A[i] - c|$$

🔴 **Định lý Toán học:** Tổng khoảng cách $\sum |A[i] - c|$ đạt giá trị nhỏ nhất khi và chỉ khi c là **Trung vị (Median)** của tập hợp $\{A[L], \dots, A[R]\}$.

4.2. Cấu trúc 2 Multisets đối xứng có điều phối cân bằng

Ta chia cửa sổ K phần tử thành 2 nửa:

- `left_set`: Chứa $\lceil K/2 \rceil$ phần tử nhỏ hơn (nửa dưới). Ta cần lấy **max** của nửa này.
- `right_set`: Chứa $\lfloor K/2 \rfloor$ phần tử lớn hơn (nửa trên). Ta cần lấy **min** của nửa này.

<code>left_set</code> [<code>x_1</code> , <code>x_2</code> , ..., <code>x_{mid}</code>] ^ * <code>left_set.rbegin()</code> (Max nửa dưới)	<code>right_set</code> [<code>x_{mid+1}</code> , ..., <code>x_K</code>] ^ * <code>right_set.begin()</code> (Min nửa trên)
--	--

Quy tắc Cân bằng Bất biến (Invariants):

- Mọi phần tử trong `left_set` đều $\leq$ mọi phần tử trong `right_set`:

$$\max(\text{left_set}) \leq \min(\text{right_set})$$

- Kích thước của `left_set` luôn bằng $\lceil K/2 \rceil$:

$$|\text{left_set}| = |\text{right_set}| + (K \& 1)$$

Duy trì Tổng để tính chi phí $O(1)$:

Ta duy trì hai biến tổng: $S_{\text{left}} = \sum_{x \in \text{left}} x$ và $S_{\text{right}} = \sum_{x \in \text{right}} x$.

Khi đó, chi phí biến đổi về trung vị $M = \max(\text{left_set})$ là:

$$\text{Cost} = (M \cdot |\text{left}| - S_{\text{left}}) + (S_{\text{right}} - M \cdot |\text{right}|)$$

Kỹ thuật Xóa an toàn trong `std::multiset`:

⚠ **Cạm bẫy sống còn:** Trong C++, nếu gọi `ms.erase(val)`, toàn bộ các phần tử có giá trị bằng `val` sẽ bị xóa sạch!

Để chỉ xóa đúng **một** phần tử rời khỏi cửa sổ, ta bắt buộc phải xóa thông qua `iterator`:

```
cpp auto it = ms.find(val); if (it != ms.end()) ms.erase(it);
```

5. Hai con trỏ kết hợp Quy hoạch động (Two Pointers DP Transitions)

5.1. Tối ưu hóa chuyển trạng thái đơn điệu

Trong bài toán Quy hoạch động 1D:

$$dp[i] = \min_{L_i \leq j \leq R_i} \{dp[j]\} + C[i]$$

Nếu hai cận L_i và R_i có tính chất **đơn điệu không giảm** theo i :

$$L_1 \leq L_2 \leq \dots \leq L_N \quad \text{và} \quad R_1 \leq R_2 \leq \dots \leq R_N$$

Thay vì quét qua tất cả $j \in [L_i, R_i]$ tốn $O(N^2)$, ta nhận thấy khoảng $[L_i, R_i]$ chính là một **Cửa sổ trượt** trên mảng dp .

- Kết hợp với **Deque đơn điệu** hoặc **SWAG Min**, ta tìm được $\min_{j \in [L_i, R_i]} dp[j]$ trong $O(1)$ **khấu hao**!
- Tổng thời gian giải quyết toàn bộ bằng DP giảm ngoạn mục từ $O(N^2)$ xuống $O(N)$.

5.2. Bài toán phân nhóm thỏa mãn độ chênh lệch cực trị

Xét bài toán: Chia mảng A thành số lượng đoạn con ít nhất sao cho mỗi đoạn con thỏa mãn $\max(A[l \dots r]) - \min(A[l \dots r]) \leq D$.

- Gọi $dp[i]$ là số đoạn tối thiểu để chia tiền tố $A[1 \dots i]$.
- Khi i tăng, vì tính chất độ chênh lệch $\max - \min$ của đoạn con dài hơn luôn $\geq$ đoạn con ngắn hơn, vị trí L_i nhỏ nhất sao cho $[L_i, i]$ thỏa mãn điều kiện cũng **tăng đơn điệu** theo i .
- Ta dùng Two Pointers kết hợp SWAG/Multiset để duy trì L_i trong $O(N)$, sau đó cập nhật:

$$dp[i] = 1 + \min_{L_{i-1} \leq j < i} dp[j]$$

6. Mảng vòng (Circular Array) & Kỹ thuật Đa con trỏ (4Sum)

6.1. Xử lý Cửa sổ trượt trên Mảng vòng

Khi mảng có tính tuần hoàn tròn (phần tử $A[N-1]$ liền kề $A[0]$), kỹ thuật chuẩn mực là **nhân đôi mảng**:

$$A' = [A_0, A_1, \dots, A_{N-1}, A_0, A_1, \dots, A_{N-1}]$$

với độ dài $2N$.

- Tiến hành chạy Hai con trỏ trên mảng A' với điều kiện siết bổ sung:

$$\text{Độ dài của sổ } (R - L + 1) \leq N$$

- Bất cứ khi nào $R - L + 1 > N$, ta bắt buộc phải ép L tăng lên ($L \leftarrow L + 1$).

- Nhờ điều kiện này, mỗi cửa sổ trên mảng vòng ban đầu được duyệt qua đúng một lần duy nhất mà không bị trùng lặp vòng thứ hai.

6.2. Kỹ thuật 4 con trỏ tối ưu $O(N^2)$ (4Sum via Two Pointers on Pairs)

Bài toán: Tìm số bộ 4 chỉ số (i, j, k, l) phân biệt sao cho $A[i] + A[j] + A[k] + A[l] = Target$.

- Tiếp cận cơ bản: Cố định 2 con trỏ ngoài $O(N^2)$ và dùng Hai con trỏ ở trong $\implies O(N^3)$.

- Tiếp cận nâng cao bằng Hai con trỏ trên tập cặp (Pair Two Pointers):

1. Sinh danh sách tất cả các cặp $(A[i] + A[j])$ với $i < j$, có kích thước $M = \frac{N(N-1)}{2} = O(N^2)$.

2. Mỗi phần tử lưu (sum, i, j) .

3. Sắp xếp danh sách cặp theo `sum` trong $O(N^2 \log N)$.

4. Chạy Hai con trỏ đối hướng (Converging Pointers) trên danh sách cặp để tìm hai cặp có tổng bằng $Target$ và kiểm tra điều kiện không giao nhau về chỉ số $(\{i, j\} \cap \{k, l\} = \emptyset)$.

5. Thời gian chạy: $O(N^2 \log N)$ (nhanh hơn $O(N^3)$ gấp hàng chục lần khi $N = 2000$).

7. Bảng Tổng hợp So sánh Kỹ thuật Cửa sổ trượt

Kỹ thuật	Phép toán áp dụng	Độ phức tạp Thời gian	Bộ nhớ phụ	Khả năng Khả nghịch	Nhận xét ứng dụng
Hai con trỏ Cơ bản	Tổng, Đếm tần số	$O(N)$	$O(1)$	Cần khả nghịch (có $\rightarrow$)	Nhanh nhất, cực kỳ gọn nhẹ
SWAG (2-Stack Queue)	GCD, Min, Max, Bitwise, Ma trận	$O(N)$ amortized	$O(N)$	Không cần khả nghịch	Giải quyết mọi Monoid tổng quát
Bitwise Window	Bitwise OR, Bitwise AND	$O(N \cdot W)$ ($W \leq 60$)	$O(W)$	Tính chất đếm số bit 1	Nhẹ hơn SWAG với phép toán bit
Two-Multisets Median	Trung vị, Thống kê thứ tự	$O(N \log K)$	$O(K)$	Bất kỳ	Tối ưu tổng khoảng cách L_1
Segment Tree Sliding	Mọi phép toán kết hợp	$O(N \log N)$	$O(4N)$	Không cần	Dễ viết nhưng hằng số lớn, TLE nếu N lớn

⚠ 8. Cạm bẫy Phòng thi & Sai lầm Thường gặp (Pitfalls)

1. **Khẩu hao bị phá vỡ trong SWAG:**
2. Nếu mỗi lần query rỗng ta lại xóa hoặc sao chép cả stack, độ phức tạp sẽ thoái hóa về $O(N^2)$.
3. *Quy tắc vàng:* Chỉ trút phần tử từ `stack_back` sang `stack_front` khi và chỉ khi `stack_front` **hoàn toàn rỗng!**
4. **Xóa nhầm toàn bộ bản sao trong `std::multiset`:**
5. Gọi `ms.erase(val)` sẽ xóa mọi phần tử có giá trị `val`.
6. Bắt buộc phải xóa bằng con trỏ: `ms.erase(ms.find(val))`.
7. **Tràn số nguyên `long long` khi tính tổng độ lệch:**
8. Khi tính tổng chi phí $\sum |A[i] - Median|$, tích $M \cdot |left|$ và các biến tổng S_{left}, S_{right} có thể vượt quá $2 \cdot 10^9$. Bắt buộc phải khai báo kiểu `long long`.
9. **Quên xử lý trường hợp cửa sổ rỗng ($L > R$):**
10. Trong quá trình co cửa sổ, con trỏ L có thể vượt quá R . Luôn đảm bảo reset trạng thái hoặc duy trì bất biến $L \leq R + 1$.

📖 9. Tài liệu Tham khảo & Đọc thêm

1. **Codeforces Edu Course:**
2. [Two Pointers Method \(Step 3: Advanced Sliding Window\)](#).
3. **Sliding Window Aggregation (SWAG):**
4. [Queue-like data structure with support for aggregate operations in \$O\(1\)\$](#) — CP-Algorithms.
5. **USACO Guide (Gold):**
6. [Sliding Window & Monotonic Queue Optimizations](#).
7. **VNOI Wiki:**
8. [Cấu trúc dữ liệu Deque và Kỹ thuật Cửa sổ trượt](#).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.